

Systèmes Embarqués et Intelligence Artificielle au plus proche des Capteurs (edge AI)

Conception des Systèmes Embarqués

Déploiement des solutions d'IA

Prise en main STM32 Cube AI

Some people call this artificial intelligence, but the reality is this technology will enhance us. — Ginni Rometty (ancienne CEO d'IBM)

Artificial intelligence is not a substitute for human intelligence. It is a tool to amplify it. — Fei-Fei Li (chercheuse, créatrice d'ImageNet)

Table des matières

Introduction du Module	3
Description	3
Objectifs	3
Compétences visées	3
Bibliographie	3
1. Comprendre l'IA	4
1.1. Ce qu'est un modèle d'IA	4
1.2. Comment fonctionne un réseau de neurones ?	4
1.3. Les deux grandes phases : apprentissage et inférence	5
1.4. Pourquoi l'IA embarquée pose des défis ?	5
1.5. Pourquoi comprendre l'IA avant de l'embarquer ?	6
2. Systèmes embarqués	7
2.1. Contraintes matérielles essentielles	8
2.2. Différences entre plateformes : F4, N6 et nouvelles générations	9
2.3. Optimiser un modèle pour l'électronique l'embarquée	9
2.4. Entraînement hors-ligne et déploiement embarqué	9
2.5. Les contraintes de consommation énergétique	10
2.6. Deux cas d'étude représentatifs	10
2.7. Une compréhension progressive des limites	10
3. IA embarquée : Sinus & Meteostat	12
3.1. Projet Sin(x) : une chaîne complète de l'entraînement au déploiement	12
3.2. Performances et limites du modèle sinus sur microcontrôleur	12
3.3. Variation des résultats selon la plateforme : F4 vs N6	13
3.4. Projet Meteostat : prédire la température à partir de données réelles	13
3.5. Comparaison des deux projets : deux pédagogies complémentaires	14
4. Discussions et analyse de la soutenabilité des solutions	16
4.1. Une réflexion émergente sur la sobriété numérique	16
4.2. Comparaison critique des deux cas étudiés	16
4.3. Principes généraux dégagés par les étudiants	17
4.4. Limites technologiques et prise de recul	17
4.5. Vers une IA embarquée responsable : pistes d'amélioration	18
5. Conclusion	19

Introduction du Module

Description

Un nouveau paradigme de conception des circuits et systèmes embarqués émerge de l'implémentation des réseaux neuronaux sur un support matériel, souvent appelée Edge AI. Pour embarquer une architecture d'IA, divers compromis sont envisageables selon le modèle de réseau de neurones et le matériel utilisé. Ce cours couvre un panel d'options mises au point dans la littérature scientifique et qui sont en train de devenir les produits de demain. Soit neuromorphique ou bio-inspirée, la solution vise à une implémentation matérielle des neurones du cortex cérébral selon leurs modèles biologiques et mathématiques. À la convergence de la microélectronique et de l'intelligence artificielle, ce module a pour objectif le développer les concepts de réseau neurone et de concevoir des systèmes embarqués d'interface capteurs sous le paradigme des circuits neuromorphiques.

Objectifs

Comprendre les architectures les plus utilisées de réseau de neurones : convolution neural network (CNN), recurrent neural network (RNN), spiking neural network (SNN). Se sensibiliser des compromis d'implémentation matériel et performance des réseaux de neurones, notamment en termes de mémoire, complexité de calcul, profondeur du réseau, et précision.

Compétences visées

Respecter les bonnes pratiques pour le déploiement de réseaux, de systèmes de communication, de solutions logicielles (TR_IMPLEMENT). Implémenter la solution dans un prototype d'IA embarqué via les cartes de développement STM32 (E_IMPLEMENT). S'organiser et collaborer efficacement dans une équipe multidisciplinaire (P_TEAM). Prendre en compte des transformations à mettre en œuvre nécessaires à la transition écologique pour un développement soutenable (P_TEDS). Structurer correctement une présentation orale de 3 min, convaincante dans le format d'un pitch pour présenter un produit (C_ORAL).

Bibliographie

Hope, T., Resheff, Y. S., & Lieder, I. (2017). *Learning TensorFlow: A Guide to Building Deep Learning Systems*. Oreilly. ISBN 9781491978511

Chollet, F. (2018). *Deep Learning with Python*. ACM. ISBN 9781617294433

Galvão, M. H., & Ferreira, P. M. (2024). AIoT Guidebook : Comprehensive tutorial for developing IoT and AI applications on STM32 microcontrollers. <https://doi.org/10.5281/zenodo.12700232>

1. Comprendre l'IA

L'intelligence artificielle (IA) est aujourd'hui largement utilisée pour analyser, reconnaître, prédire ou décider à partir de données. Dans les systèmes embarqués, elle prend une place croissante, car elle permet d'ajouter des capacités autonomes là où les approches classiques reposaient presque exclusivement sur des traitements déterministes. Malgré la diversité des applications modernes — reconnaissance d'images, classification de signaux, détection d'anomalies ou prévision — les étudiants soulignent tous qu'un principe fondamental demeure : **l'IA n'est pas un ensemble de règles programmées explicitement, mais un apprentissage automatique réalisé à partir d'exemples.**

1.1. Ce qu'est un modèle d'IA

On comprend un modèle d'IA comme un système mathématique qui apprend à associer des entrées à des sorties. Au lieu d'indiquer précisément comment résoudre un problème, on fournit au modèle un ensemble de données, et celui-ci ajuste progressivement ses paramètres pour réduire son erreur.

Trois éléments essentiels reviennent dans toutes les formulations :

1. **Les données**, qui constituent la matière première (images, signaux, couples entrée-sortie...).
2. **L'architecture**, c'est-à-dire la structure du modèle : couches, neurones, poids, fonctions d'activation.
3. **La fonction de perte**, qui mesure l'erreur et guide l'optimisation.

Le rôle du modèle n'est pas de « mémoriser », mais de **généraliser**. Un modèle trop simple échoue à apprendre (sous-apprentissage). Un modèle trop complexe devient lourd et prédit mal hors des exemples vus (surapprentissage).

1.2. Comment fonctionne un réseau de neurones ?

On comprend le réseau de neurones comme un enchaînement d'opérations simples. Chaque neurone :

- a. reçoit des valeurs en entrée,
- b. les combine avec des poids,
- c. applique une transformation (fonction d'activation),
- d. et transmet le résultat aux neurones de la couche suivante.

Chaque couche « fait un petit bout du travail ». En se succédant, ces transformations permettent d'extraire progressivement des motifs ou des tendances, et d'approximer des relations de plus en plus complexes.

Des notions récurrentes apparaissent dans les rapports :

- a. **poids** : valeurs ajustées durant l'apprentissage,
- b. **activation** : non-linéarité permettant au réseau de modéliser des phénomènes complexes,
- c. **erreur, perte** : écart entre prédiction et réalité,
- d. **prédiction** : sortie produite par le modèle.

Ces concepts permettent de comprendre qu'un réseau n'est finalement qu'un enchaînement de calculs, mais capable de représenter des comportements difficiles à coder manuellement.

1.3. Les deux grandes phases : apprentissage et inférence

Les rapports insistent fortement sur la séparation entre **apprentissage** et **inférence**, un point essentiel en IA embarquée.

Phase d'apprentissage

- a. Toujours réalisée sur un ordinateur puissant, jamais sur la carte microcontrôleur.
- b. Le modèle voit un grand nombre d'exemples et compare ce qu'il prédit à la réponse attendue.
- c. La rétropropagation ajuste les poids pour diminuer progressivement la fonction de perte.
- d. L'entraînement utilise des notions organisationnelles comme :
 - I. **batches** : groupes de données utilisés pour une mise à jour,
 - II. **epochs** : nombre de passages du modèle sur l'ensemble des données.

Cette phase peut être longue et demande des bibliothèques spécialisées.

Phase d'inférence

- a. Le modèle est converti dans une version compacte (par exemple TensorFlow Lite).
- b. Il est chargé dans la mémoire d'un microcontrôleur.
- c. Le modèle devient **figé** : il ne réapprend rien.
- d. La carte se contente d'effectuer les calculs nécessaires pour donner une prédiction.

L'inférence doit être **rapide, fiable et économe en énergie**, particulièrement sur les plateformes STM32 utilisées par les étudiants.

1.4. Pourquoi l'IA embarquée pose-t-elle des défis ?

On relève plusieurs difficultés récurrentes :

- a. la nécessité de choisir un modèle ni trop simple ni trop complexe,
- b. la vérification de la capacité du modèle à généraliser,

- c. l'impact du nombre de neurones ou de couches sur les performances,
- d. la sensibilité aux choix des données d'entrée,
- e. les limites révélées même sur des problèmes simples comme **$\sin(x)$** .

Pour évaluer ces limites, l'erreur **RMS** est citée comme indicateur central. Elle permet de mesurer de manière objective la justesse du modèle en inférence.

1.5. Pourquoi comprendre l'IA avant de l'embarquer ?

Il est important de comprendre comment l'IA apprend et comment elle fait ses prédictions avant de s'intéresser à son déploiement sur microcontrôleur. On constate que :

- a. les microcontrôleurs imposent des contraintes strictes de calcul, mémoire et consommation,
- b. les modèles doivent être simplifiés sans perdre trop de précision,
- c. les approches IA peuvent être pertinentes ou non selon les tâches.

À travers des cas d'usage tels que l'approximation du sinus ou la prédiction simple de séries temporelles, ils découvrent les compromis entre **précision**, **vitesse** et **efficacité énergétique**, ce qui aide à comprendre quand l'IA embarquée devient réellement intéressante.

2. Systèmes embarqués

L'étude des systèmes embarqués amène à comprendre que les microcontrôleurs fonctionnent dans un environnement radicalement différent des ordinateurs classiques. Les rapports soulignent tous que ces cartes disposent de ressources très limitées : mémoire réduite, fréquence modeste, unités de calcul simplifiées et absence d'accélération matérielles complexes. L'objectif d'un microcontrôleur n'est pas d'exécuter des modèles lourds, mais de **consommer peu, rester réactif et fonctionner longtemps**, souvent dans des conditions temps réel strictes.

La **NUCLEO-F446RE**, quant à elle, intègre un Cortex-M4 cadencé à 180 MHz, avec 128 Ko de RAM et 512 Ko de Flash. Elle est très populaire dans la communauté des développeurs embarqués grâce à son bon compromis entre performance et coût, et elle dispose également de nombreuses interfaces de communication.

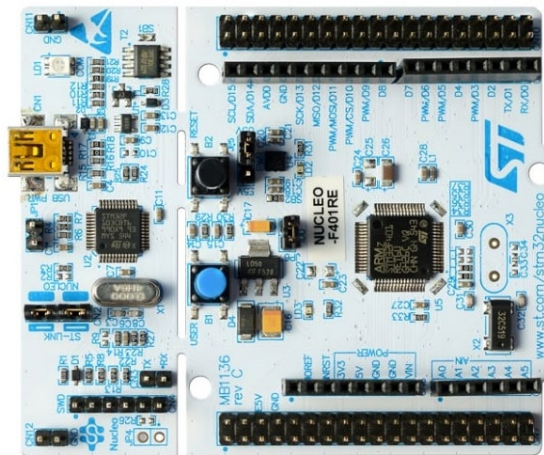


Fig. Carte de développement **NUCLEO-F446RE**

La **NUCLEO-N657X0** est basée sur un microcontrôleur ARM Cortex-M33 cadencé jusqu'à 160 MHz, avec une mémoire RAM de 320 ko et une mémoire Flash de 512 Ko. Elle offre des interfaces variées, par exemple UART, SPI, I2C, et dispose de ressources hardware avancées, comme un accélérateur cryptographique et neuromorphic processing unit (NPU), qui peuvent être utiles pour des applications embarquées sophistiquées. La NPU intégrée sur la NUCLEO-N657X0 dispose d'un accélérateur Neural-ART (600 GOPS, 1 GHz), optimisé pour l'inférence de modèles CNN, avec streaming dédié, décompression dynamique et sécurité intégrée. La disponibilité de la NPU transforme le STM32N657 en un MCU capable d'IA hautes performances, comparable à des microprocesseurs tout en restant dans les contraintes très faibles de consommation et de coût d'un microcontrôleur.



Fig. Carte de développement **NUCLEO-N657X0**

Ces caractéristiques techniques – fréquence de processeur, mémoire disponible, et interfaces – sont essentielles pour choisir la carte la plus adaptée à l’implémentation d’un modèle d’IA embarqué, notamment en raison des contraintes de calcul et de mémoire qu’impose l’exécution d’un réseau de neurones.

2.1. Contraintes matérielles essentielles

Le compromis à être relevé est très concret : « mémoire très limitée », « consommation à surveiller », « désactiver des GPIO inutiles », « temps d’exécution à la microseconde ». Cette formulation directe permet de transmettre intuitivement le fonctionnement d’un système embarqué : **chaque octet compte, chaque instruction compte**, et les marges d’erreur sont réduites.

Sur une carte comme la **NUCLEO-F446RE** (Cortex-M4), les contraintes typiques mentionnées sont:

- a. RAM : quelques centaines de ko
- b. Flash : quelques centaines de ko à quelques Mo
- c. Fréquence modérée, parfois sans floating point unit (FPU) ou avec une FPU simple précision
- d. Pas d’accélération IA dédiée
- a. Dans ces conditions, embarquer un réseau de neurones impose de :
- e. limiter le nombre de poids,
- f. maîtriser les buffers intermédiaires,
- g. respecter la latence exigée par l’application (ex. <10 ms),
- h. appliquer une quantification agressive, souvent en **int8**.

La difficulté consiste à trouver un équilibre : réduire trop le modèle dégrade ses performances, mais le laisser trop complexe le rend impossible à embarquer.

2.2. Différences entre plateformes : F4, N6 et nouvelles générations

La série **NUCLEO-N6**, par exemple, dispose de ressources bien plus élevées :

- a. plusieurs Mo de RAM disponible dans la carte de développement NUCLEO,
- b. Flash plus large,
- c. unité NPU pour le calcul amélioré des opérations avec vecteurs, matrices, tensors,
- d. accélération matérielle plus riche pour les opérations vectorielles.

Conséquence : il devient possible d'utiliser des modèles plus profonds, des couches supplémentaires ou des architectures plus coûteuses, tout en conservant une latence d'inférence acceptable.

L'arrivée d'un **NPU (Neural Processing Unit)** marque une étape importante. Un NPU exécute de manière accélérée les opérations propres au deep learning (produits matriciels, convolutions, activations). Dans ce cas, la question n'est plus seulement "peut-on faire tourner un modèle ?", mais "quelle complexité maximale peut-on embarquer ?". De petites CNN ou modèles temporels deviennent ainsi envisageables sur microcontrôleur.

2.3. Optimiser un modèle pour l'électronique l'embarquée

Il existe plusieurs techniques permettant d'adapter un modèle aux contraintes embarquées :

- a. **Quantification** : réduction de la précision numérique (ex. float32 $\rightarrow$ int8). Cela diminue l'occupation mémoire et accélère les calculs sur du matériel limité.
- b. **Élagage (pruning)** : suppression de connexions peu utiles. Le réseau devient plus léger et les prédictions demandent moins d'opérations.
- c. **Architectures spécifiques** : utilisation de modèles compacts adaptés aux ressources réduites, exploitant des opérations à faible coût.

L'enjeu est toujours le même : **trouver le meilleur compromis entre précision et ressources disponibles.**

2.4. Entraînement hors-ligne et déploiement embarqué

Un point souligné systématiquement est que :

1. *l'entraînement se fait hors-ligne, sur PC ou serveur,*
2. *le microcontrôleur n'exécute que l'inférence.*

La procédure typique est la suivante :

- a. Entraîner un modèle avec PyTorch ou TensorFlow.
- b. Obtenir des poids figés.
- c. Exporter le modèle dans une forme optimisée (TensorFlow Lite Micro, CMSIS-NN...).
- d. Charger ce modèle sur la carte, qui réalise uniquement la prédiction.

Cette séparation réduit fortement la charge de calcul embarquée et rend l'IA utilisable sur microcontrôleur.

2.5. Les contraintes de consommation énergétique

On rappelle que la consommation est directement liée à :

- a. la fréquence des prédictions,
- b. la taille du modèle,
- c. la durée des calculs.

Une exécution continue consomme davantage, alors qu'une prédiction ponctuelle peut être rendue très économique. Là encore, la pertinence du modèle dépend du contexte : **l'IA ne doit être utilisée que si elle apporte un bénéfice clair par rapport à une méthode déterministe.**

2.6. Deux cas d'étude représentatifs

Les projets proposés incluent deux scénarios complémentaires :

- **Approximation du sinus** Un réseau simple déployé sur STM32F446RE, permettant d'observer directement comment un modèle minimal se comporte sous contrainte.
- **Prédiction de précipitations** Modèle plus complexe, utilisant trois paramètres météorologiques, déployé sur STM32N6, illustrant la transition vers des architectures plus puissantes.

L'analyse de ces deux cas met en évidence les compromis entre précision (erreur RMS), temps d'exécution et consommation.

2.7. Une compréhension progressive des limites

À travers ces expérimentations, on découvre que :

- un réseau trop complexe peut saturer la mémoire,
- un modèle trop simple ne généralise pas,
- la fréquence d'échantillonnage impose une latence maximale,
- les optimisations sont obligatoires, pas optionnelles.

De manière intuitive, ils comprennent que **moins il y a de ressources disponibles, plus le modèle doit être simple**, et que l'IA embarquée est une pratique d'ingénierie, pas une simple transposition des méthodes serveur.

3. IA embarquée : Sinus & Meteostat

Dans cette partie, on découvre deux projets concrets qui permettent de comprendre les enjeux réels de l'intelligence artificielle embarquée : l'approximation de la fonction sinus et la prédiction météorologique à partir de données Meteostat. Bien que ces tâches paraissent simples, elles deviennent extrêmement instructives dès qu'elles sont déployées sur un microcontrôleur. Elles mettent en évidence non seulement la logique d'entraînement hors-ligne et de déploiement embarqué, mais surtout les différences radicales entre un modèle exécuté sur PC et un modèle exécuté sur une carte STM32.

3.1. Projet Sin(x) : une chaîne complète de l'entraînement au déploiement

Le premier projet consiste à approximer la fonction $\sin(x)$ à l'aide d'un petit réseau de neurones. L'architecture du réseau des neurones à être entraîné peut-être illustré de la façon suivante.

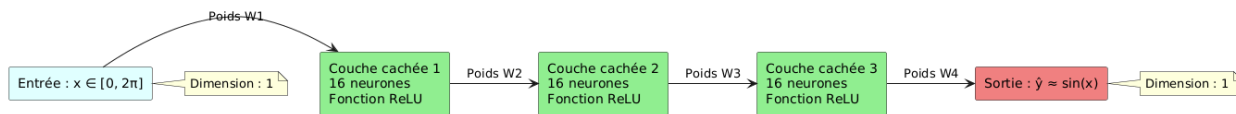


Fig. Architecture de réseau des neurones du modèle sinus

On suit une chaîne de développement complète :

- Génération des données : création de points équidistants sur l'intervalle $[0, 2\pi]$.
- Entraînement hors-ligne : construction et apprentissage d'un Multi-Layer Perceptron (MLP) sur PC avec TensorFlow (TF) ou Keras.
- Conversion du modèle : exportation en format allégé TFLite.
- Intégration via X-CUBE-AI : import du modèle dans STM32CubeIDE.
- Instrumentation embarquée :
- configuration d'un timer à $1 \mu s$ pour mesurer la latence,
- envoi des résultats via UART,
- désactivation de composants inutiles pour optimiser la consommation.

Le retour d'expérience des étudiants adopte un ton pratique, parfois narratif : « une fois le code téléversé... on voit la valeur apparaître dans le terminal ». Ce style rend la démarche concrète et reproductible dans ce cadre pédagogique.

Le projet sinus joue un rôle fondamental : il permet de vérifier que *toute la chaîne IA embarquée fonctionne*, même avec un modèle simple. La prédiction obtenue à partir de $\sin(2.0)$, par exemple, se rapproche bien de la valeur exacte, illustrant que le microcontrôleur peut correctement exécuter un réseau de neurones minimal.

3.2. Performances et limites du modèle sinus sur microcontrôleur

L'un des enseignements majeurs du projet est la comparaison entre deux méthodes :

Méthode classique : utilisation directe de la fonction trigonométrique du microcontrôleur, optimisée matériellement.

Méthode IA : inférence d'un réseau de neurones embarqué.

On observe que :

- a. la méthode classique consomme environ 17 mA,
- b. la méthode IA peut monter à 36 mA ou davantage,
- c. la latence est plus élevée pour le réseau de neurones,
- d. l'erreur RMS reste faible (par exemple 0,0136),
- e. mais le coût énergétique par point est nettement supérieur.

En d'autres termes, l'IA fonctionne, mais consomme plus et calcule plus lentement qu'un simple $\sin()$. Ce contraste souligne une idée centrale de l'IA embarquée : un modèle peut être performant et précis sur le Cloud, tout en étant lourd et coûteux sur une carte à ressources limitées.

3.3. Variation des résultats selon la plateforme : F4 vs N6

On pourrait tester également l'approximation du sinus sur des cartes plus puissantes, comme la NUCLEO-N6. Les résultats montrent que :

- a. la N6 réduit la latence,
- b. l'inférence est plus fluide,
- c. la quantification est moins contraignante,
- d. et certaines optimisations matérielles limitent la consommation.

Cependant, même avec une plateforme renforcée, le retour d'expérience des étudiants souligne que l'IA n'apporte pas d'avantage fonctionnel pour un problème simple comme $\sin(x)$, dont la solution analytique est déjà disponible.

3.4. Projet Meteostat : prédire la température à partir de données réelles

Le second projet utilise des données Meteostat pour prédire une valeur météorologique future, souvent la température. Ce projet déplace l'attention de l'étudiant : la difficulté n'est plus dans l'IA elle-même, mais dans le prétraitement des données. L'architecture du réseau des neurones à être entraîné peut-être illustrée de la façon suivante.

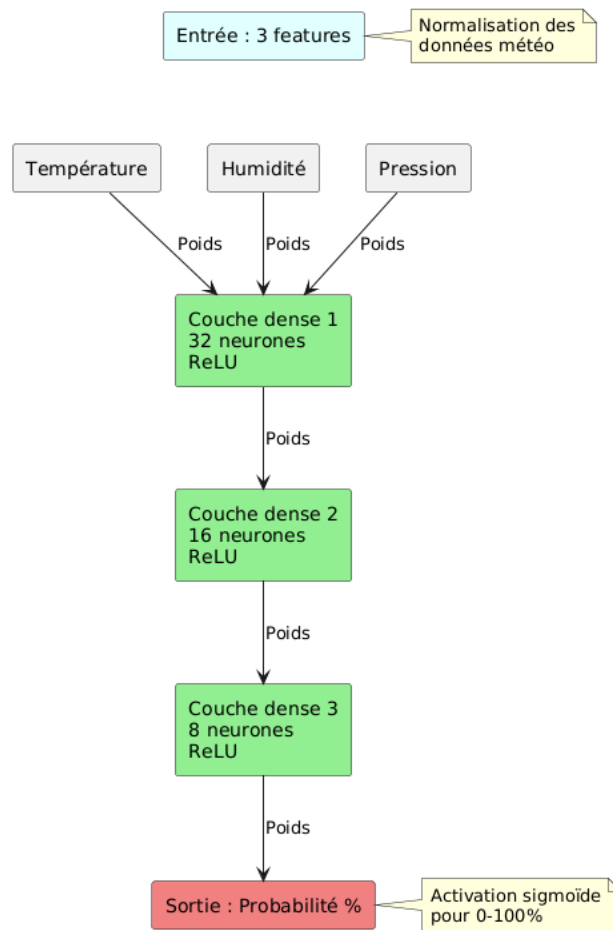


Fig. Architecture de réseau des neurones du modèle de précipitation à partir de la base meteostat

Les étapes principales sont les suivantes :

- Collecte de données réelles via API Meteostat (température, humidité, pression...).
- Nettoyage et normalisation des mesures.
- Construction d'une fenêtre temporelle (ex. $24\text{ h} \times 4\text{ variables}$).
- Entraînement d'un modèle simple adapté à la taille des entrées.
- Validation et conversion du modèle au format embarqué.
- Déploiement sur STM32, souvent sur une F446 ou une N6.

On réalise rapidement que : ce n'est pas le réseau de neurones qui est difficile à construire, mais la préparation des données. Le cas Meteostat illustre la pertinence réelle de l'IA : contrairement au sinus, la relation entre température, humidité et pression n'est pas analytique, et l'IA peut offrir ici un avantage.

3.5. Comparaison des deux projets : deux pédagogies complémentaires

Ces deux projets en apparence simple permettent de mettre en lumière plusieurs enseignements essentiels :

1. Le sinus sert d'exemple "laboratoire". Il montre clairement :
 - a. comment entraîner,
 - b. comment convertir,
 - c. comment déployer,
 - d. et comment mesurer latence, RMS et consommation.
2. Meteostat, au contraire, montre :
 - a. comment manipuler un jeu de données réel,
 - b. comment structurer des entrées temporelles,
 - c. et comment valider un modèle dans des conditions proches d'un usage réel.
3. En comparant les deux projets, on observe :
 - a. un même réseau peut être instantané sur PC, mais lent sur carte,
 - b. un modèle léger peut consommer deux fois plus d'énergie qu'une solution classique,
 - c. un problème trivial comme $\sin(x)$ révèle les limites de l'IA embarquée,
 - d. la préparation des données est souvent plus difficile que le modèle lui-même.

4. Discussions et analyse de la soutenabilité des solutions

Les expérimentations menées autour des projets sinus et Meteostat amènent naturellement à s'interroger sur la pertinence et la soutenabilité des approches basées sur l'intelligence artificielle en contexte embarqué. Même lorsque cela n'était pas explicitement demandé, de nombreuses remarques émergent spontanément : « pas pertinent », « trop lent », « consomme beaucoup pour juste un sinus ». Ces commentaires traduisent une prise de conscience de l'impact énergétique et matériel des modèles IA, surtout lorsque leur utilité réelle est limitée.

4.1. Une réflexion émergente sur la sobriété numérique

À travers leurs essais, on découvre que l'IA embarquée peut introduire des coûts élevés :

- a. consommation énergétique accrue,
- b. temps d'exécution parfois multiplié par 20 ou 30,
- c. occupation mémoire plus importante,
- d. complexité de la chaîne d'intégration.

Ces constats amènent une question centrale : **pourquoi utiliser un réseau de neurones pour résoudre un problème qui possède déjà une solution mathématique simple ?**

L'exemple du sinus est parlant : la fonction est disponible nativement dans le microcontrôleur, optimisée matériellement, et bien plus rapide que l'inférence d'un modèle. L'IA apparaît ici comme une solution « surdimensionnée » qui consomme davantage pour un résultat moins précis.

On comprend alors un principe fondamental : **la sobriété numérique dépend des choix de conception autant que de la technologie utilisée.**

4.2. Comparaison critique des deux cas étudiés

Les deux projets mettent en évidence des situations où l'IA est justifiée et d'autres où elle ne l'est pas.

Projet sinus : la solution classique reste supérieure

Les résultats présentés dans les rapports montrent des écarts clairs :

Précision : l'algorithme classique est exact, le réseau de neurones comporte une erreur RMS (ex. 0,03).

Temps d'exécution : jusqu'à 20 fois plus rapides pour la méthode classique.

Consommation : réduction notable (par exemple 108 mW contre 153 mW pour l'IA).

Ce cas rappelle que, **si une formule analytique existe**, l'IA n'apporte aucun avantage fonctionnel, mais génère un coût matériel et énergétique important.

Dans le cas de la prédiction météo, on observe :

- a. un **gain de précision** de l'ordre de quelques pourcents pour le modèle IA,
- b. un **temps d'inférence nettement plus élevé**,
- c. une **consommation supérieure**,
- d. mais une capacité à capturer des relations non linéaires que la régression simple ne modélise pas.

Ici, l'IA a une légitimité, mais uniquement si :

- a. la précision supplémentaire est réellement utile,
- b. les contraintes d'énergie et de temps réel le permettent,
- c. le modèle est adapté à la plateforme.

Ce contraste entre les projets illustre bien que **la pertinence de l'IA dépend avant tout de la complexité du problème**, et non du simple fait qu'un modèle peut être déployé sur une carte.

4.3. Principes généraux dégagés par les étudiants

À partir des différents retours, plusieurs idées fortes se dégagent :

L'IA n'est pas magique : elle n'améliore pas systématiquement un système.

Certaines tâches n'ont aucun intérêt à être résolues par un réseau de neurones.

La performance doit être évaluée avec des métriques objectives : RMS, latence, consommation.

Chaque choix d'architecture a un coût : taille du modèle, fréquence d'échantillonnage, buffers intermédiaires, mis en veille...

La consommation énergétique devient une métrique centrale, notamment dans des dispositifs alimentés sur batterie.

L'embarquabilité exige une approche holistique : au-delà des performances brutes, il faut considérer la maintenance, l'actualisation des modèles et la robustesse en situation réelle.

Les étudiants prennent ainsi conscience que la question n'est pas simplement **Peut-on embarquer un modèle IA ?**, mais **Est-ce réellement utile et soutenable ?**.

4.4. Limites technologiques et prise de recul

La prise de recul sur les transformations nécessaires à la transition écologique pour un développement soutenable soulignent les limites actuelles de l'écosystème :

- a. outils encore en évolution ou trop récentes,
- b. complexité de la chaîne d'intégration IA,

- c. coût mémoire et énergétique importante,
- d. difficulté à optimiser un réseau pour une plateforme contrainte,
- e. exigences de compétences avancées en IA, optimisation et embarqué.

Ces limites renforcent l'idée que **les réseaux de neurones ne sont pas naturellement compatibles avec l'extrême frugalité des microcontrôleurs**, sauf dans des cas bien ciblés. La réflexion va parfois plus loin : dans un contexte où les ressources matérielles deviennent plus coûteuses, où la consommation énergétique est un enjeu global, et où les microcontrôleurs restent limités par nature, la pertinence d'utiliser une IA pour des tâches simples est remise en question.

4.5. Vers une IA embarquée responsable : pistes d'amélioration

Les étudiants identifient plusieurs leviers pour rendre l'IA embarquée plus soutenable :

- a. **Quantification et réduction de précision** (int8, int16).
- b. **Pruning et compression** pour supprimer les poids non essentiels.
- c. **Architectures compactes** (réseaux spécialisés pour l'embarqué).
- d. **Exploitation d'accélérateurs matériels** (NPU, DSP optimisés).
- e. **Mise en veille intelligente** entre les inférences.
- f. **Approches hybrides** combinant IA et méthodes classiques.
- g. **Optimisation multiobjective** prenant en compte précision, latence et énergie.

Ces pistes montrent que la performance brute n'est pas l'unique priorité : **l'efficacité énergétique devient une contrainte de conception à part entière.**

5. Conclusion

Au terme de ces travaux, on retient que l'intelligence artificielle embarquée n'est ni une solution miraculeuse ni une technologie à rejeter en bloc. Elle apparaît comme un outil parmi d'autres, dont la pertinence dépend avant tout de la nature du problème à résoudre. Les projets menés montrent que l'IA peut fonctionner sur microcontrôleur, mais qu'elle apporte un bénéfice réel seulement lorsque les relations à modéliser sont suffisamment complexes pour justifier son coût en ressources.

L'un des apports majeurs de ces expériences réside dans la méthode adoptée : expérimenter, mesurer, comparer puis analyser. En suivant cette démarche, nous avons pu observer de manière objective les forces et les limites des approches IA et classiques. Cette façon de travailler, centrée sur l'évaluation concrète plutôt que sur l'idée préconçue qu'une technologie est « meilleure », constitue une base solide qu'ils pourront réutiliser dans leurs futurs projets, quel que soit le domaine.

Les conclusions issues des différents rapports convergent : l'IA doit être utilisée avec discernement. Sa mise en œuvre implique des coûts en temps de calcul, en mémoire et en énergie, qui ne sont pas négligeables dans les systèmes embarqués. Lorsqu'une solution analytique ou déterministe existe déjà, comme dans le cas du sinus, l'IA se révèle souvent moins efficace et moins sobre. À l'inverse, pour des problèmes plus riches comme la prédiction météorologique, elle peut offrir une amélioration mesurable, à condition d'accepter les compromis nécessaires.

Cette prise de recul amène les étudiants à comprendre que la sobriété numérique fait partie intégrante du processus de conception. Ils identifient que la taille du modèle, sa fréquence d'exécution, la qualité des données et les contraintes matérielles doivent être considérées conjointement. L'IA embarquée devient alors un équilibre entre intelligence du système, efficacité énergétique, simplicité des solutions et contraintes réelles du matériel.

En définitive, on conclut que l'important n'est pas d'utiliser l'IA parce qu'elle est moderne, mais parce qu'elle est appropriée. Cette capacité à choisir une solution pour de bonnes raisons, en connaissant ses bénéfices comme ses limites, représente l'apprentissage le plus durable du module.

Sujet de TP 1 : Problème MNIST

Le jeu de données **MNIST** est un benchmark classique en apprentissage automatique pour la classification d'images. Il contient des images en niveaux de gris de chiffres manuscrits (0 à 9). Chaque image est de taille **28×28 pixels**, soit 784 pixels au total. L'objectif est de prédire correctement la classe (le chiffre) associée à chaque image parmi les **10 classes** possibles (0–9). La figure 1 illustre ce jeu de données.

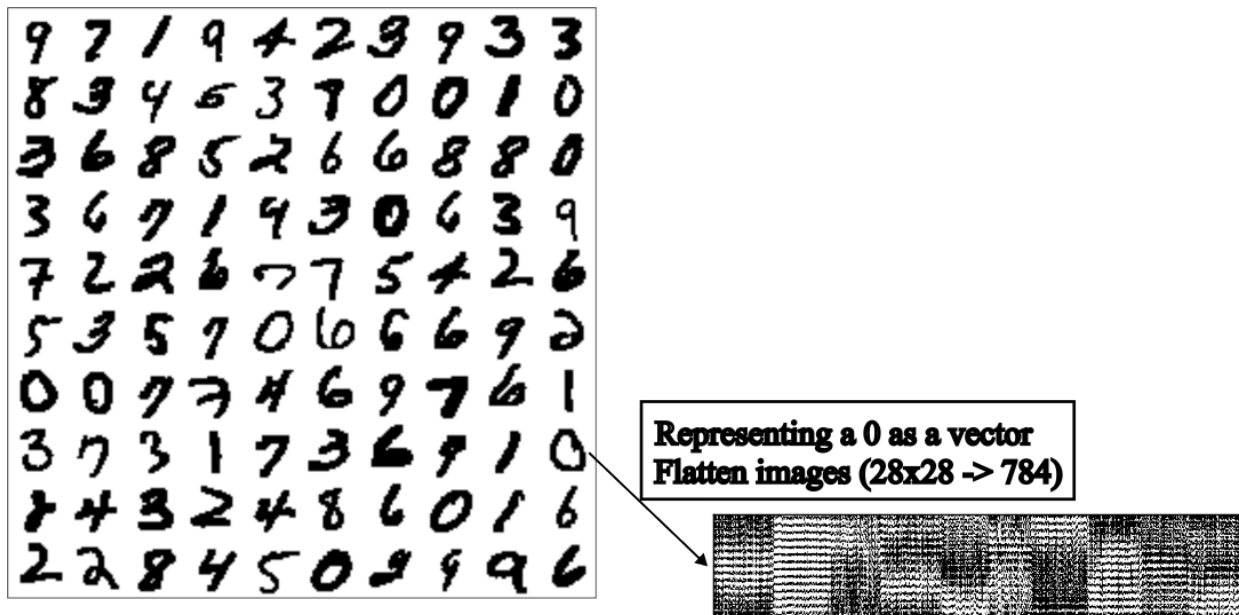


Figure 1. Dataset du problème MNIST et représentation vectorielle des 784 pixels.

La nomenclature utilisée est souvent w_i^j , où w est un vecteur ; i est l'indice du vecteur ; j est la classe que w représente. Enfin, W est l'ensemble des vecteurs $[w^0, \dots, w^9]$, correspondant aux classes disponibles.

Pour entraîner un réseau de neurones avec la base MNIST, chaque image $[28 \times 28]$ est transformée en un vecteur aplati $[1 \times 784]$. Ainsi, la position spatiale des pixels n'est plus explicitement exploitée dans le modèle. On propose d'implémenter un modèle de réseau des neurones denses, aussi connu comme *multi-layer perceptron* (MPL) ou *fully connected*. Il doit être composé :

- d'une couche d'entrée de dimension 784,
- d'une couche de sortie dense de 10 neurones, produisant une distribution de probabilité sur les 10 classes,
- d'un nombre des couches cachées différentes selon le besoin.

Chaque sortie représente la probabilité que l'image appartienne à une classe donnée. L'apprentissage pourra être réalisé par descente de gradient en minimisant l'entropie croisée (*cross-entropy loss*).

1. Choix de la Fonction d'Activation

Assumer l'optimiseur **adam** et la fonction coût **categorical_crossentropy** pour l'étude de la fonction d'activation.

- Implémenter ce réseau des neurones avec deux couches (entrée et sortie) à partir de la fonction d'activation **softmax**
- Implémenter ce réseau des neurones avec 2 ou 3 couches cachées et **ReLU** comme fonction d'activation.
- Implémenter ce réseau des neurones avec 1 ou 2 couches cachées et **tanh** comme fonction d'activation.
- Implémenter ce réseau des neurones avec 1 ou 2 couches cachées et **sigmoid** comme fonction d'activation.
- Comparer les différentes implémentations de MPL selon l'architecture et fonction d'activation. Justifier les compromis entre : **accuracy**, nombre d'**epochs**, nombre de synapses (dimension de W normalement stocké en mémoire).

2. Choix de l'Algorithme d'Optimisation

Choisir la meilleure architecture de réseau des neurones dans l'ensemble des critères présentés pour faire varier les implémentations précédentes selon les optimiseurs suivants :

Optimizer	Compromis des Algorithmes
SGD	Base simple, classique, intuition sur descente de gradient
Adam	Moderne, converge vite, combine momentum + RMSprop
RMSprop	Adaptation du taux d'apprentissage, stabilité sur entrées variées
Adagrad	Learning rate par paramètre, illustration de l'effet adaptatif

3. Choix de la Fonction Coût

Le choix de la fonction de coût dépend du type de problème et de la sortie du modèle. Ajuster le modèle de manière conséquente pour pouvoir varier les implémentations précédentes selon les fonctions suivantes :

Type de Problème	Sortie du Modèle	Meilleure Fonction Coût
Binary Classification (ex. spam / non spam)	Probabilité $0 \leq y \leq 1$	binary_crossentropy
Multi-Class Classification (ex. chien / chat / lapin)	Vecteur des probabilités de chaque classe	categorical_crossentropy <i>À utiliser si les labels sont encodés en one-hot (ex. [0,0,1,0,...])</i>
Multi-Class (One Label at a Time)	Logits (raw scores)	sparse_categorical_crossentropy <i>À utiliser si les labels sont des entiers (ex. 0,1,2,...). Pratique pour MNIST si les chiffres sont codés 0-9</i>

Sujet de TP 2 : Interface Capteur & STM32

La carte de développement **NUCLEO-N657X0** est basée sur un microcontrôleur ARM Cortex-M33 cadencé jusqu'à 160 MHz, avec une mémoire RAM de 320 ko et une mémoire Flash de 512 Ko. Elle est illustrée sur la figure 1. Elle offre des interfaces variées et dispose de ressources hardware avancées, comme un accélérateur cryptographique et neuromorphic processing unit (NPU) type Neural-ART (600 GOPS, 1 GHz), optimisé pour l'inférence de modèles CNN. La carte capteurs **X-NUCLEO-IKS01A3** regroupe plusieurs capteurs MEMS permettant de mesurer différentes grandeurs physiques telles que l'accélération, le champ magnétique, la température, l'humidité ou encore la pression atmosphérique. Ces capteurs communiquent avec le microcontrôleur via un bus I2C, largement utilisé dans les systèmes embarqués pour connecter des périphériques à faible débit. Elle est illustrée sur la figure 2.



Fig. 1. Carte de développement **NUCLEO-N657X0**



Fig. 2. Carte de développement **X-NUCLEO-IKS01A3**

1. LED Blink sur NUCLEO-STM32N657

Utiliser le logiciel STM32 Cube IDE pour programmer cette carte avec le tutoriel Blink LED qui est disponible sur : [Wiki](#).

- Allumer les trois LEDs (rouge, vert, bleu).
- Implémenter un chenillard avec ces trois LEDs et une temporalisation de 3 secondes.
- Imprimer sur la console les états de fonctionnement de la carte : <début d'application>, <début de chenillard>, <ROUGE>, <VERT>, <BLEU>, <début de chenillard>, ... Pour démontrer l'utilisation des logs sur interface console.

2. Interface Capteurs

L'objectif de ce travail pratique est de comprendre comment intégrer des drivers matériels fournis par le constructeur, configurer une interface I²C sur le microcontrôleur et implémenter les fonctions bas niveau permettant la lecture et l'écriture dans les registres des capteurs. Les étudiants devront ensuite utiliser ces drivers pour acquérir des données issues des capteurs et les afficher via l'interface série du microcontrôleur. Ce travail permettra ainsi de se familiariser avec la programmation embarquée, la communication avec des périphériques externes et l'utilisation de bibliothèques logicielles fournies par les fabricants. Ils sont disponibles sur https://github.com/STMicroelectronics/STMems_Standard_C_drivers une bibliothèque de drivers pour les capteurs sur la carte d'interface capteur identifiés par :

- **LSM6DSO** : 3-axis accelerometer + 3-axis gyroscope
 - **LIS2MDL** : 3-axis magnetometer
 - **LIS2DW12** : 3-axis accelerometer
 - **HTS221** : humidity and temperature sensor
 - **LPS22HH** : pressure sensor
 - **STTS751** : temperature sensor
- Proposer une implémentation sur **NUCLEO-STM32N657** capable de lire les valeurs des trois capteurs. Imprimer les valeurs lues comme un log sur l'interface console.
 - Modifier l'utilisation des LEDs de l'exercice précédent pour qu'ils deviennent un repère visuel de l'état de fonctionnement de la carte, soit
 - <ROUGE> - Occupé en train de lire les capteurs
 - <VERT> - Disponible et en attente d'une prochaine lecture

Étapes de réalisation (procédure recommandée)

1. Récupération des drivers

- Cloner / télécharger STMems_Standard_C_drivers depuis le dépôt indiqué.
- Dans le dossier du driver, repérer pour chaque capteur les fichiers xxx_reg.c et xxx_reg.h.
- Copier les paires xxx_reg.c / xxx_reg.h nécessaires dans un nouveau sous-dossier de votre projet (ex. Drivers/STMems/).
- Ajouter ce dossier au include path du projet.

2. Étudier l'exemple

- Le dossier exemple du dépôt contient des templates montrant comment appeler `read_reg` / `write_reg` et comment effectuer une lecture de capteur. Lisez-les pour comprendre l'API fournie par `xxx_reg.c/h`.

3. Implémenter les fonctions bas-niveau

- Les fichiers `xxx_reg.c` appellent des fonctions bas-niveau `read_reg` / `write_reg` (non implémentées par ST). Vous devez les remplacer par des appels I²C HAL adaptés à votre MCU.
- Exemple-type (à adapter à vos noms d'instances et à la gestion d'adresse 7/8 bits) :

```

<> C
int32_t platform_read(void *handle, uint8_t reg, uint8_t *buf, uint16_t len) {
    // handle peut être un pointeur vers l'I2C handle (ex. &hi2c2)
    I2C_HandleTypeDef *hi2c = (I2C_HandleTypeDef*) handle;
    uint16_t dev_addr = /* adresse 7-bit du capteur << 1 si HAL demande 8-bit */;

    if (HAL_I2C_Mem_Read(hi2c, dev_addr, reg, I2C_MEMADD_SIZE_8BIT, buf, len, 1000) == HAL_OK)
        return 0;
    else
        return -1;
}

```

- Implémentez symétriquement `platform_write` avec `HAL_I2C_Mem_Write`.
- **Remarque importante** : vérifiez si les adresses données dans la doc sont 7-bit ; HAL attend souvent l'adresse 8-bit (7-bit << 1). Testez et documentez votre choix.

4. Configurer le bus I²C (CubeMX)

- Créez/générez `MX_I2C2_Init()` (ou autre interface choisie).
- Exemple de mapping fourni : PA12 -> I2C2_SCL, PA11 -> I2C2_SDA (adaptez si votre carte diffère).
- Sélectionnez vitesse (standard/fast) en fonction du capteur et de votre câblage. CubeMX génère les fichiers d'initialisation.

5. Activer la sortie flottante pour printf (optionnel mais utile)

- Si vous souhaitez afficher des float via `sprintf` / `printf`, activez l'option `_printf_float` dans les paramètres du compilateur (Project Settings > MCU Settings dans CubeIDE).

6. Tester la communication

- Avant de lire les mesures, testez la présence des périphériques I²C :
- Tenter une lecture d'un registre connu (ex. WHO_AM_I) ou implémenter une boucle de scan d'adresses I²C.
- Si les lectures échouent : vérifier les pull-ups I²C, câblage, adresses, et initialisation HAL.

7. Lire et afficher les grandeurs

- Utiliser les fonctions fournies par `xxx_reg.c` (ex. `stts751_temperature_raw_get` ou équivalent) pour obtenir les valeurs brutes, puis appliquer les conversions (fourni dans les drivers).

- Afficher via UART/printf : température (°C), humidité (%RH), pression (hPa), accélération (mg / m/s²), magnétisme (μT), etc.

3. Réseau Ethernet

La carte de développement **NUCLEO-N657X0** possède un connecteur RJ45 classique pour une connexion à un réseau locale. En revanche, les protocoles de communication exigent un système d'exploitation temps réelle adapté à la carte.

- Proposer une implémentation du système d'exploitation FreeRTOS sur la carte **NUCLEO-N657X0**
- Implémenter un LWIP (stack IP) avec DHCP + ICMP et utiliser le Socket BSD dans la configuration de l'API/
- Exécuter un ping sur google.com
- Modifier l'utilisation des LEDS de l'exercice précédent pour qu'ils deviennent un repère visuel de l'état de fonctionnement de la carte, soit
 - <BLEU> - Communication réseau
- Imprimer sur la console les états de fonctionnement de la carte : <début d'application>, <Lecture Interface Capteurs>, <Communication Réseau>, <Lecture Interface Capteurs>, <Communication Réseau>, ... Pour démontrer l'utilisation des logs sur interface console.

Étapes de réalisation (procédure recommandée)

1. Préparer l'environnement

- Vérifier la compatibilité de la carte cible (NUCLEO-F746ZQ ou STM32F746G-DISCO) avec la pile LWIP et FreeRTOS fournis.
- Ouvrir CubeMX / CubeIDE, sélectionner la carte et générer un projet de base (clock, périphériques ethernet si présents).

2. Choix de l'API FreeRTOS (CMSISv1 vs CMSISv2)

- Décider d'utiliser CMSISv1 ou CMSISv2. CMSISv2 est fonctionnel mais consomme plus de pile.
- **Si vous choisissez CMSISv2** : augmenter la pile du thread qui reçoit les paquets (ethernetif_input) pour éviter des débordements.
- Exemple d'ajustement à appliquer dans ethernetif.c (ou là où le thread est créé) :

```
</> C  
  
memset(&attributes, 0x0, sizeof(osThreadAttr_t));  
attributes.name = "EthIf";  
attributes.stack_size = 1024; // augmenter si débordement (ex. 2048)  
attributes.priority = osPriorityRealtime;  
osThreadNew(ethernetif_input, netif, &attributes);
```

- *Conseil* : si vous observez encore des stack overflow après 1024, essayez 1536 ou 2048 et reliez cela à l'utilisation mémoire totale du système (tasks, heap).

3. Configuration LWIP (mémoire et logs)

- Ouvrir la configuration LWIP dans CubeMX et augmenter la taille mémoire globale :
 - MEM_SIZE → **5000 bytes** (valeur recommandée initiale). Ajuster si nécessaire selon la charge (buffers, sockets ouverts).

- Activer les logs pour diagnostiquer :
 - Dans le fichier lwipopts.h ajouter (dans la zone USER CODE) :

```

</> C
/* USER CODE BEGIN 0 */
#define LWIP_DEBUG
/* USER CODE END 0 */
    
```

- Recompiler et vérifier que les messages LWIP apparaissent sur la console (UART). Sinon, vérifier la configuration printf / retargeting.

4. Résoudre les problèmes de CRC sur les paquets

- Si les pings ICMP montrent des réponses mais avec CRC fail sur le replay, le calcul CRC matériel peut ne pas être correctement configuré. Vérifier dans CubeMX :
 - Dans **Middleware > LWIP** : activer **CRC by hardware**, désactiver les autres options de CRC.
 - Dans **Configuration MCU > CRC** (ou périphériques) : activer l'option **Computing > CRC** (activer le périphérique CRC dans CubeMX).
- Re-générer, recompiler et retester le ping.

5. Tests et validations

- Démarrer l'interface réseau et effectuer des tests simples :
 - ping depuis une machine sur le même réseau vers l'adresse IP de la carte.
 - Ouvrir une socket BSD (socket, bind, listen / connect) et échanger de petites données (ex. echo simple).
- Observer les logs LWIP et FreeRTOS pour détecter erreurs (timeouts, mem errors, stack overflow).
- En cas d'échec, isoler :
 - Vérifier la configuration PHY (autonegotiation / duplex / speed) si applicable.
 - Vérifier l'initialisation du netif (adresse IP, netmask, gateway).
 - Vérifier les callbacks d'interruption et que le cache/invalidations (si CPU avec cache) sont correctement gérés lors des DMA.

6. Optimisations et robustesse

- Ajuster MEM_SIZE si vous ouvrez plusieurs sockets simultanément.
- Ajuster tailles de piles des tasks réseau si vous avez de nombreuses connexions concurrentes.
- Mettre en place une gestion des erreurs LWIP (callbacks d'allocation, logs) pour redémarrer proprement l'interface si besoin.

Sujet de TP 3 : Cloud versus Edge AI

ThingSpeak est une plateforme d'analyse IoT dans le cloud qui permet de collecter, visualiser et exploiter des flux de données en temps réel provenant d'appareils connectés. Le service offre des graphiques instantanés, l'exécution d'analyses en ligne et le traitement des données à la volée, ce qui en fait un outil adapté au prototypage et aux preuves de concept IoT. Les données peuvent être envoyées via des API REST ou MQTT. De plus, les intégrations cloud-to-cloud (illustré sur la figure 1) avec The Things Network, Senet, la passerelle Libelium Meshlium et Particle.io permettent aux données des capteurs d'atteindre ThingSpeak via les protocoles de communication Ethernet, Wifi, 4G/3G, comme aussi les réseaux basse consommation comme le LoRaWAN®. ThingSpeak permet aussi le stockage sans infrastructure serveur, la création d'alertes e-mail basées sur des événements issus des capteurs connectés. Dans le marché actuel, ils existent des offres sur [Google](#), [Microsoft](#), [Amazon](#). Dans la suite des TPs, nous allons supposer une offre académique disponible par l'université ou gratuite selon la motivation de l'étudiant.

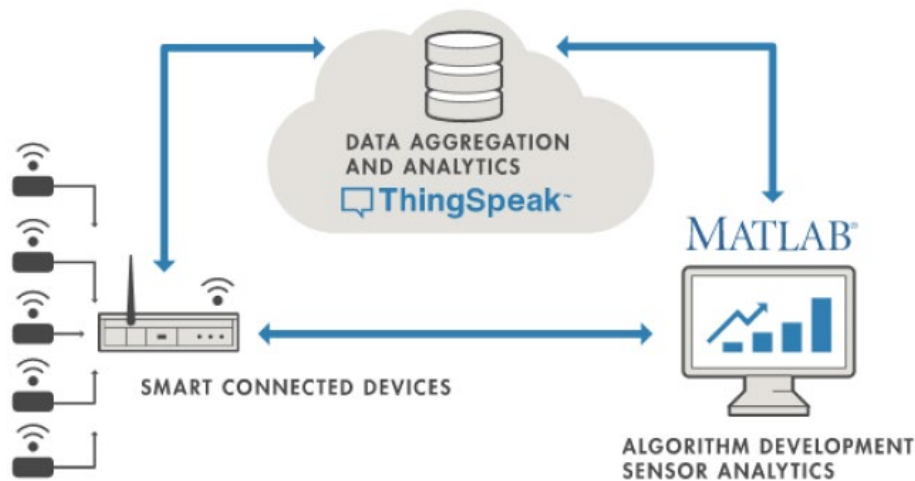


Fig. 1. Illustration du service d'agrégation des données sur le Cloud de MatWorks.

L'**IA Cloud** désigne l'utilisation de serveurs distants haute performance pour stocker, traiter et analyser des données provenant d'objets connectés. Contrairement à l'IA embarquée, cette approche centralise l'intelligence sur des plateformes comme **ThingSpeak**, permettant d'exécuter des algorithmes complexes qui dépassent les capacités matérielles des appareils locaux.

Les avantages stratégiques de cette architecture incluent :

- **Puissance de calcul massive** : Capacité d'exécuter des modèles de Deep Learning gourmands en ressources et de traiter des pétaoctets de données.
- **Agrégation de données (Big Data)** : Centralisation des informations provenant de milliers de capteurs pour identifier des corrélations globales.
- **Stockage historique illimité** : Conservation de données sur le long terme pour l'entraînement et l'amélioration continue des modèles.
- **Mises à jour centralisées** : Possibilité de modifier un algorithme instantanément pour l'ensemble du parc d'appareils sans intervention physique.
- **Visualisation avancée** : Utilisation d'outils comme MATLAB au sein de ThingSpeak pour générer des tableaux de bord et des analyses statistiques complexes.

L'**IA embarquée** désigne l'intégration de capacités d'intelligence artificielle au sein d'appareils ou de systèmes, souvent appelés **applications de bord** (*edge AI*). Cela permet d'exécuter des tâches de machine learning localement, sans nécessiter de ressources de calcul externes telles que les services cloud.

Les principaux avantages de cette approche sont les suivants :

- **Latence réduite** : Grâce à la proximité de l'unité de traitement par rapport à la tâche.
- **Faible consommation de bande passante** : Besoins minimaux en transmission de données.
- **Faible consommation d'énergie** : Optimisée pour l'efficacité énergétique.
- **Indépendance vis-à-vis du réseau** : Fonctionne sans connectivité Internet constante.
- **Haute performance** : Capable de gérer efficacement des tâches exigeantes.

Pour maximiser l'efficacité d'un système intelligent, il ne s'agit pas de choisir entre l'IA embarquée et le Cloud, mais de les faire collaborer. Cette approche, souvent orchestrée via des outils comme **MATLAB** et **ThingSpeak**, repose sur une répartition stratégique de l'intelligence. Le véritable avantage de ce parallèle réside dans la boucle de rétroaction entre les deux systèmes :

- **L'IA Embarquée agit** : Le capteur surveille une machine en temps réel et prend des micro-décisions de sécurité.
- **Le Cloud apprend** : Les données significatives sont envoyées à **ThingSpeak**. Là, des algorithmes MATLAB puissants analysent les données de milliers de machines pour détecter des modèles d'usure invisibles à l'échelle locale.
- **L'IA s'améliore** : Une fois le nouveau modèle optimisé dans le Cloud, il est renvoyé vers l'appareil embarqué sous forme de mise à jour. L'appareil devient alors plus intelligent grâce à l'expérience partagée de toute la flotte.

1. Entrainement de Modèle d'IA pour le problème MeteoStat

La bibliothèque Python [Meteostat](#) offre un moyen simple et efficace d'accéder à des données météorologiques et climatiques ouvertes en s'appuyant sur la bibliothèque Pandas. Elle permet de récupérer facilement des observations historiques et des statistiques climatiques prêtes à être analysées sous forme de tableaux de données. Les informations proviennent de l'interface de données massives de Meteostat, qui centralise et harmonise des jeux de données issus de nombreuses sources publiques, principalement des agences gouvernementales.

- Entraîner en Python/Tensorflow un modèle de IA pour la classification météorologique. Choisir des classes du système de classification météo standard à partir de :

Ciel / couverture nuageuse

-  Clair / ensoleillé (clear sky)
-  Peu nuageux (mostly sunny)
-  Partiellement nuageux (partly cloudy)
-  Nuageux / couvert (overcast)

Précipitations

-  Pluie
-  Averses
-  Neige

-  Neige légère / averses de neige
-  Pluie et neige mêlées

Phénomènes particuliers

-  Orage
 -  Brouillard / brume
 -  Vent fort
 -  Orage violent
- b. Etudier le compromis entre le nombre de classes disponibles, la taille du modèle, précision du modèle (accuracy). Justifier sur le hyperparamètres choisi pour l'entraînement.

2. Neural Network dans le Cloud MatWorks

L'objectif de cette étape est d'implémenter un réseau des neurones sur le Cloud de MatWorks. Pour cela, nous allons utiliser un modèle déjà entrainer avec Python/Tensorflow qui serait déployer sur le Cloud avec ThingSpeaks. En passant par Open Neural Network Exchange (ONNX), on utilise un format intermédiaire propre et standardisé : TensorFlow est d'abord converti en un graphe unifié, puis MATLAB lit ce graphe simplifié. ONNX est indépendant des frameworks, centré sur l'inférence et basé sur des couches et opérations standardisées, ce qui le rend plus simple à interpréter. MATLAB manipule ainsi un réseau de neurones clair et structuré plutôt que de devoir gérer toute la complexité de l'écosystème TensorFlow.

ONNX est généralement plus fiable car il s'agit d'un format de modèle neutre et standardisé, spécialement conçu pour faciliter le transfert de modèles d'intelligence artificielle entre différents frameworks. Lors d'une importation directe d'un modèle TensorFlow vers MATLAB, le logiciel doit effectuer de nombreuses opérations complexes : analyser le graphe *SavedModel*, reconstruire les couches du réseau, traduire les opérations propres à TensorFlow en couches MATLAB et gérer les métadonnées spécifiques à Keras. De plus, les modèles TensorFlow contiennent souvent des éléments dépendants du framework, comme des opérations spécifiques, des couches personnalisées, des nœuds utiles uniquement pendant l'entraînement ou encore des formats de graphe variables selon les versions. Cette richesse fonctionnelle rend la conversion plus fragile et plus sujette aux erreurs. ONNX a été créé par Microsoft et Meta avec un objectif précis : assurer l'interopérabilité entre les outils d'intelligence artificielle. Il sert de langage commun permettant de transférer de manière fiable des modèles entraînés d'un environnement à un autre.

- a. Exporter le modèle d'IA déjà entrainer en Python/Tensorflow. Par exemple, nous pouvons s'en servir des résultats de la question précédent. N'oubliez pas de transférer le modèle TensorFlow ->ONNX->MatLab. L'exemple MNIST du TP1 serait implémenter de la façon suivante :

<> Python



Run

```
import tf2onnx
import tensorflow as tf

model = tf.keras.models.load_model("mnist_softmax_model")
spec = (tf.TensorSpec((None,784), tf.float32, name="input"),)
output_path = "mnist_softmax.onnx"

model_proto, _ = tf2onnx.convert.from_keras(model, input_signature=spec, output_path=output_path)
```

<> MATLAB



```
model = importONNXNetwork("mnist_softmax.onnx", ...
    OutputLayerType="classification");
```

- b. Implémenter un script MatLab qui tourne le réseau des neurones pour le problème MeteoStat en mode inférence. C'est-à-dire exécution d'une classification à partir du réseau déjà entrainer. Un exemple MNIST donnerait ceci

<> MATLAB



```
% Load network
model = importONNXNetwork("mnist_softmax.onnx", OutputLayerType="classification");

% Read image
img = imread("digit.bmp");

if size(img,3) == 3
    img = rgb2gray(img);
end

img = im2single(img);
img = reshape(img, 1, []);

% Predict
scores = predict(model, img);

% Get digit
[prob, digit] = max(scores);
digit = digit - 1;

% Print
fprintf("Predicted digit: %d (confidence %.2f%%)\n", digit, prob*100);
```

- c. Utiliser les données réelles obtenus sur la carte STM32 (voir TP2) et qui sont disponibles sur ThingSpeaks. Sauvegarder le résultat de la classification sur un canal spécifique.
- d. Utiliser l'API Talkback pour faire afficher le résultat de la classification sur la carte STM32.

3. Neural Network dans la carte STM32N6

L'objectif de cette étape est d'implémenter un réseau des neurones sur la carte STM32N6. Un exemple d'implémentation d'un modèle simple est disponible sur le [GitHub](#). Il sera nécessaire d'implémenter un code de réseau de neurones en mode inférence à partir des données acquises avec les capteurs de la carte fille.

- a. Exporter le modèle d'IA déjà entrainer en Python/Tensorflow. Par exemple, nous pouvons s'en servir des résultats de la question précédent. N'oubliez pas de transférer le modèle TensorFlow ->ONNX->C (via X Cube AI)
- b. Implémenter un code C qui tourne le réseau des neurones pour le problème MeteoStat en mode inférence. C'est-à-dire exécution d'une classification à partir du réseau déjà entrainer.
- c. Utiliser les données réelles acquises via les capteurs de la carte fille. Comparer ce résultat en termes de accuracy et temps d'exécutions avec la solution Cloud AI (ThingSpeaks).
- d. Mesurer ou estimer la consommation de puissance de la carte lors d'une action de classification via le réseau implémenté. Pour cela, il faut désactiver les GPIOs et tout composant non essentiel. Préparer un programme témoins qui a toutes les étapes sauf l'exécution du modèle d'IA. Exécuter le programme témoin et le programme avec le réseau en mesurant le courant d'alimentation du jumper correspondant. (PS. Lisez bien la documentation de la carte avant cette procédure et venez discuter avec les charges de TP pour valider la manipulation).

Sujet de TP 4 : Examen de TP en Compétences

L'évaluation du module ETS923_ESET est sous la forme d'examen de TP approche par compétences. Vous seriez évalués sur les critères suivants :

1. Respecter les bonnes pratiques pour le déploiement de réseaux, de systèmes de communication, de solutions logicielles (TR_IMPLEMENT).
2. Implémenter la solution dans un prototype d'IA embarqué via les cartes de développement STM32 (E_IMPLEMENT).
3. S'organiser et collaborer efficacement dans une équipe multidisciplinaire (P_TEAM).
4. Prendre en compte des transformations à mettre en œuvre nécessaires à la transition écologique pour un développement soutenable (P_TEDS).
5. Structuré correctement une présentation orale de 3 min, convaincante dans le format d'un pitch pour présenter un produit (C_ORAL).

Préparez-vous à répondre les questions des chargés de TP lors du passage. Ayez une démonstration du système prête et clés en main, car elle est nécessaire pour la validation des compétences implémentation. Votre travail doit être rendu sur la forme de GitHub, lien unique pour votre équipe, avec un dépôt sur Moodle. Les codes déposés et les commentaires seront corrigés après cette session pour ajuster la note finale. Les meilleurs GitHub feront partie des annales pour les années suivantes. Une action de communication sur le sujet sera proposée avec un rendu sous la forme de vidéo, produite sur place à partir de la démonstration du prototype. Selon votre autorisation d'image, cette vidéo sera diffusée lors des forums et salons de promotion de la formation. La vidéo sera aussi disponible dans un service public de diffusion des vidéos en format ouvert.